UNSUPERVISED LEARNING AND NEURAL NETWORKS

24ITT52 – MACHINE LEARNING

MINI PROJECT REPORT

Submitted by

NIRANJAN G

24ITR085

in partial fulfilment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY



(Autonomous)

PERUNDURAI, ERODE – 638 060

OCTOBER 2026

ABSTRACT

This mini project demonstrates the concepts of Unsupervised Learning and Artificial Neural Networks through practical implementation using Python. The primary objective is to understand how machine learning techniques can identify hidden patterns in data and how neural networks learn by adjusting their internal parameters. The unsupervised learning part focuses on customer segmentation using a dataset containing 500 customer records with features such as age, annual income and spending score. The dataset is preprocessed and standardized to ensure that the features contribute appropriately to the clustering process. Four clustering algorithms, namely K-Means, K-Medoids, Hierarchical Clustering and Density-Based Spatial Clustering of Applications with Noise (DBSCAN), are implemented on the same dataset to compare their approaches to grouping similar customers. The resulting clusters are visualized using scatter plots, while a dendrogram illustrates the hierarchical merging process in agglomerative clustering. The Silhouette Score is used to evaluate and compare clustering quality wherever applicable, and the number of clusters and detected noise points are examined to understand the behaviour of each algorithm. The neural network part demonstrates a feedforward artificial neural network consisting of an input layer, a hidden layer and an output layer. The sigmoid activation function is used to calculate neuron outputs, and forward propagation is performed to generate the predicted output. Backpropagation calculates the error gradients and updates the weights and biases to improve the network's output over successive training epochs. The prediction error is recorded across 10 epochs to observe the learning process and understand how parameter updates influence the network's performance. Python libraries such as Pandas, NumPy, Matplotlib and Scikit-learn are used for data preprocessing, algorithm implementation, visualization and evaluation. The project provides practical exposure to different clustering techniques, hierarchical relationships, noise detection, neural network architecture and iterative learning mechanisms. Overall, it establishes a foundation for understanding unsupervised pattern discovery and neural network training, along with their potential applications in customer analysis, data mining, pattern recognition and intelligent decision-making systems.

UNSUPERVISED LEARNING

Unsupervised Learning is a type of machine learning in which a model identifies hidden patterns, similarities and structures in data without using predefined target labels. It helps discover meaningful groups and relationships within a dataset. In this project, unsupervised learning is applied to customer segmentation using K-Means, K-Medoids, Hierarchical Clustering and DBSCAN. These algorithms group customers based on their age, annual income and spending score.

ARTIFICIAL NEURAL NETWORKS (ANN)

An Artificial Neural Network is a computational model inspired by the structure of biological neural networks. It consists of interconnected artificial neurons arranged in input, hidden and output layers. Each connection has a weight, and neurons use activation functions to process information and produce outputs. During training, the network adjusts its weights and biases to minimize prediction error. In this project, a feedforward neural network is implemented using the sigmoid activation function, forward propagation and backpropagation to demonstrate how the network calculates outputs and updates its parameters over 10 epochs.

OBJECTIVES

1. To understand the fundamental concepts of unsupervised learning and artificial neural networks.
2. To implement K-Means, K-Medoids, Hierarchical Clustering and DBSCAN for customer segmentation.
3. To visualize clusters and evaluate clustering performance using the Silhouette Score.
4. To demonstrate forward propagation and backpropagation in a feedforward neural network.
5. To observe how updating weights and biases reduces prediction error over successive epochs.

METHODOLOGY

The project is divided into two phases. In the first phase, a customer segmentation dataset (from kaggle) containing 500 records is loaded, and the features Age, Annual Income and Spending Score are standardized to ensure fair comparison. Four clustering algorithms, namely K-Means, K-Medoids, Hierarchical Clustering and DBSCAN, are applied to identify groups of customers with similar characteristics. A dendrogram illustrates the hierarchical merging process. The clustering performance is compared using the Silhouette Score wherever applicable, and the number of clusters and noise points are analysed.

In the second phase, a feedforward Artificial Neural Network (ANN) with three input neurons, two hidden neurons and one output neuron is implemented. The sigmoid activation function is used to calculate neuron outputs. Forward propagation generates the predicted output, which is compared with the target value to calculate the error. Backpropagation then updates the weights and biases using a learning rate of 0.9. The network is trained for 10 epochs, and the prediction error is recorded at each epoch to observe how the network learns and improves its output. These two phases demonstrate the practical application of clustering algorithms and the basic learning mechanism of neural networks.

PHASE I - UNSUPERVISED LEARNING

Dataset Link :
https://drive.google.com/file/d/1M1iLmwvjFfjI62IO43KjIAO_Y4EtFLJt/view?usp=sharing

	A	B	C	D	E
1	CustomerI	Age	AnnualIncome	SpendingScore	
2	1	23	16.7	87	
3	2	26	9.4	68	
4	3	23	22.5	80	
5	4	19	32	87	
6	5	22	34	84	
7	6	19	28	71	
8	7	26	24.6	78	
9	8	19	34.8	79	
10	9	20	22.2	85	
11	10	23	28.3	84	
12	11	31	21.7	75	
13	12	19	29.9	90	
14	13	22	18.3	73	
15	14	25	30.9	85	
16	15	19	26.9	81	
17	16	23	32	82	
18	17	25	25.5	83	
19	18	25	13.3	77	
20	19	20	19.9	78	
21	20	28	18.1	89	
22	21	18	22.3	81	
23	22	24	30.7	87	
24	23	21	21.3	88	
25	24	21	14.8	70	
26	25	18	29	81	
27	26	25	21.6	81	

customer_segmentation

Common setup:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from google.colab import files
uploaded = files.upload()
df = pd.read_csv("customer_segmentation.csv")
# Select the three features
X = df[["Age", "AnnualIncome", "SpendingScore"]]
# Standardize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
# 2D representation for plotting
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Dataset shape:", df.shape)
display(df.head())
Saving customer_segmentation.csv to customer_segmentation.csv
Dataset shape: (500, 4)
```

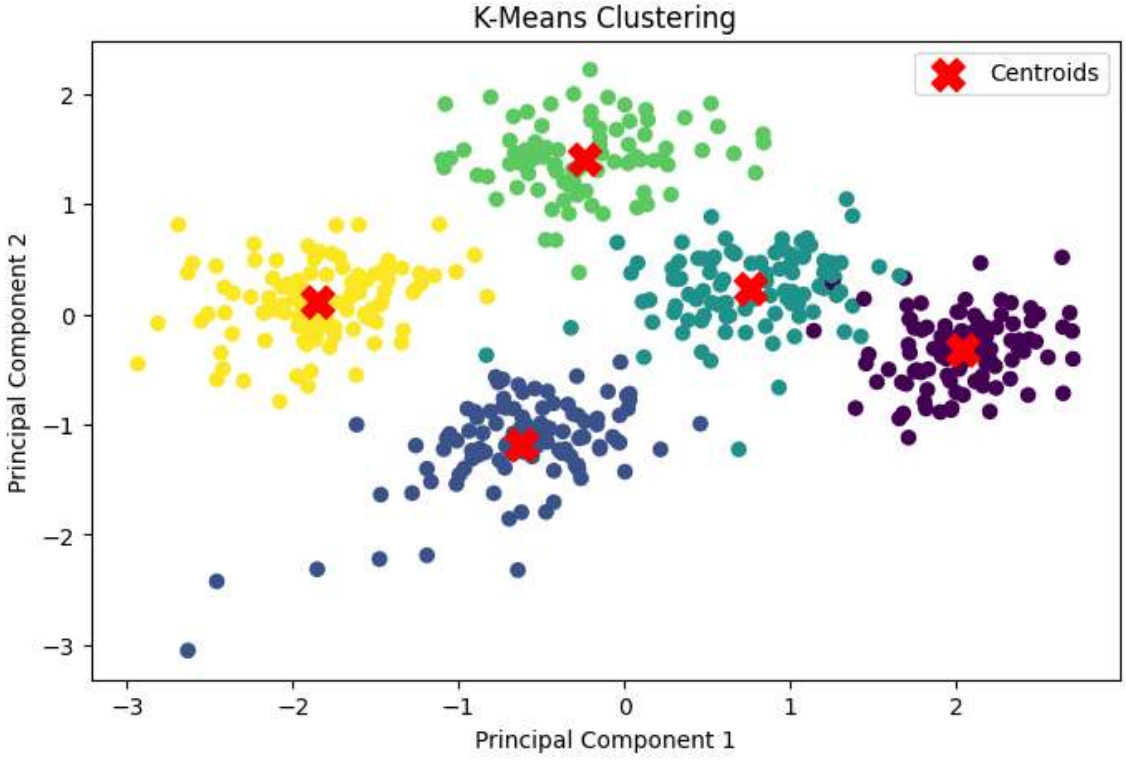
	CustomerID	Age	AnnualIncome	SpendingScore
0	1	23	16.7	87
1	2	26	9.4	68
2	3	23	22.5	80
3	4	19	32.0	87
4	5	22	34.0	84

K-Means clustering:

```
from sklearn.cluster import KMeans
kmeans = KMeans(
    n_clusters=5,
    init="random",
    n_init=10,
    random_state=42
)
df["KMeans"] = kmeans.fit_predict(X_scaled)
# Display cluster assignments
print(df[["CustomerID", "KMeans"]].head(10))
print("\nCluster counts:")
print(df["KMeans"].value_counts().sort_index())
# Plot clusters
plt.figure(figsize=(8, 5))
plt.scatter(
    X_pca[:, 0], X_pca[:, 1],
    c=df["KMeans"], cmap="viridis", s=35
)
centers_pca = pca.transform(kmeans.cluster_centers_)
plt.scatter(
    centers_pca[:, 0], centers_pca[:, 1],
    marker="X", s=200, c="red", label="Centroids"
)
plt.title("K-Means Clustering")
```

```
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.show()
print("SSE:", round(kmeans.inertia_, 3))
```

```
   CustomerID  KMeans
0           1       0
1           2       0
2           3       0
3           4       0
4           5       0
5           6       0
6           7       0
7           8       0
8           9       0
9          10       0
Cluster counts:
KMeans
0    100
1    107
2    101
3     88
4    104
Name: count, dtype: int64
```



SSE: 238.521

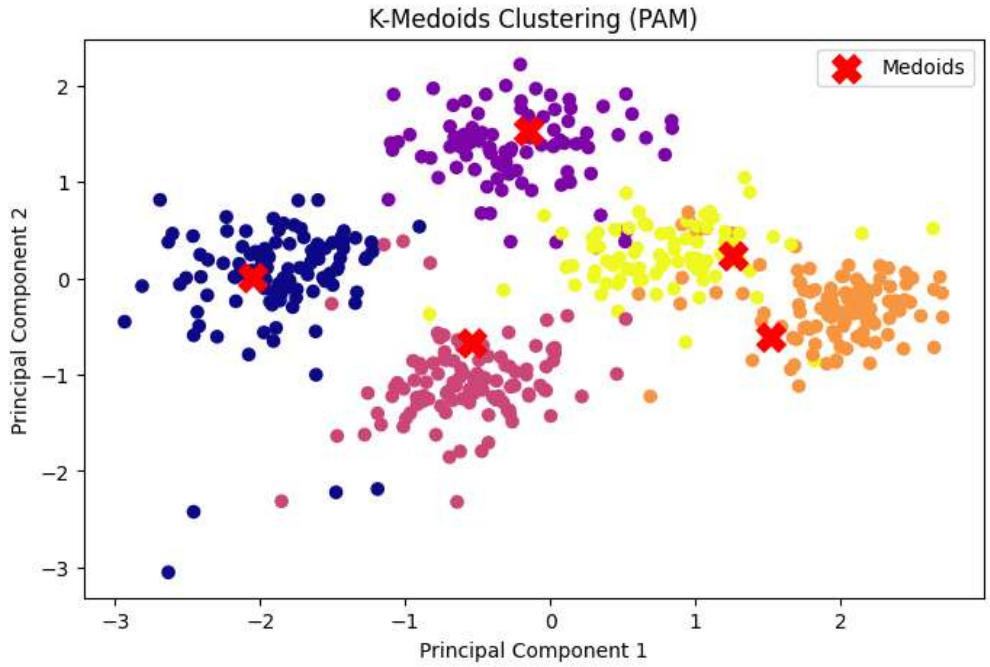
K-Medoids clustering:

```
from sklearn.metrics import pairwise_distances
def k_medoids_pam(X, k=5, max_iter=20, random_state=42):
    rng = np.random.default_rng(random_state)
    # Pairwise Manhattan distances
    D = pairwise_distances(X, metric="manhattan")
    n = len(X)
    medoids = rng.choice(n, size=k, replace=False)
    def calculate_cost(selected):
        return np.min(D[:, selected], axis=1).sum()
    cost = calculate_cost(medoids)
    for iteration in range(max_iter):
        improved = False
        for i in range(k):
            for candidate in range(n):
                if candidate in medoids:
                    continue
                new_medoids = medoids.copy()
                new_medoids[i] = candidate
                new_cost = calculate_cost(new_medoids)
                if new_cost < cost:
                    medoids = new_medoids
                    cost = new_cost
                    improved = True
                    break
            if improved:
                break
        if not improved:
            break
    labels = np.argmin(D[:, medoids], axis=1)
    return labels, medoids, cost
labels, medoids, cost = k_medoids_pam(X_scaled, k=5)
df["KMedoids"] = labels
print("Medoid customer IDs:")
print(df.iloc[medoids]["CustomerID"].tolist())
print("\nCluster counts:")
print(df["KMedoids"].value_counts().sort_index())
plt.figure(figsize=(8, 5))
plt.scatter(
    X_pca[:, 0], X_pca[:, 1],
    c=labels, cmap="plasma", s=35
)
plt.scatter(
    X_pca[medoids, 0], X_pca[medoids, 1],
    marker="X", s=200, c="red", label="Medoids"
)
```

```
plt.title("K-Medoids Clustering (PAM)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.show()
print("Total Manhattan distance cost:", round(cost, 3))
```

Medoid customer IDs:
[162, 459, 326, 45, 217]

Cluster counts:
KMedoids
0 104
1 94
2 108
3 109
4 85
Name: count, dtype: int64



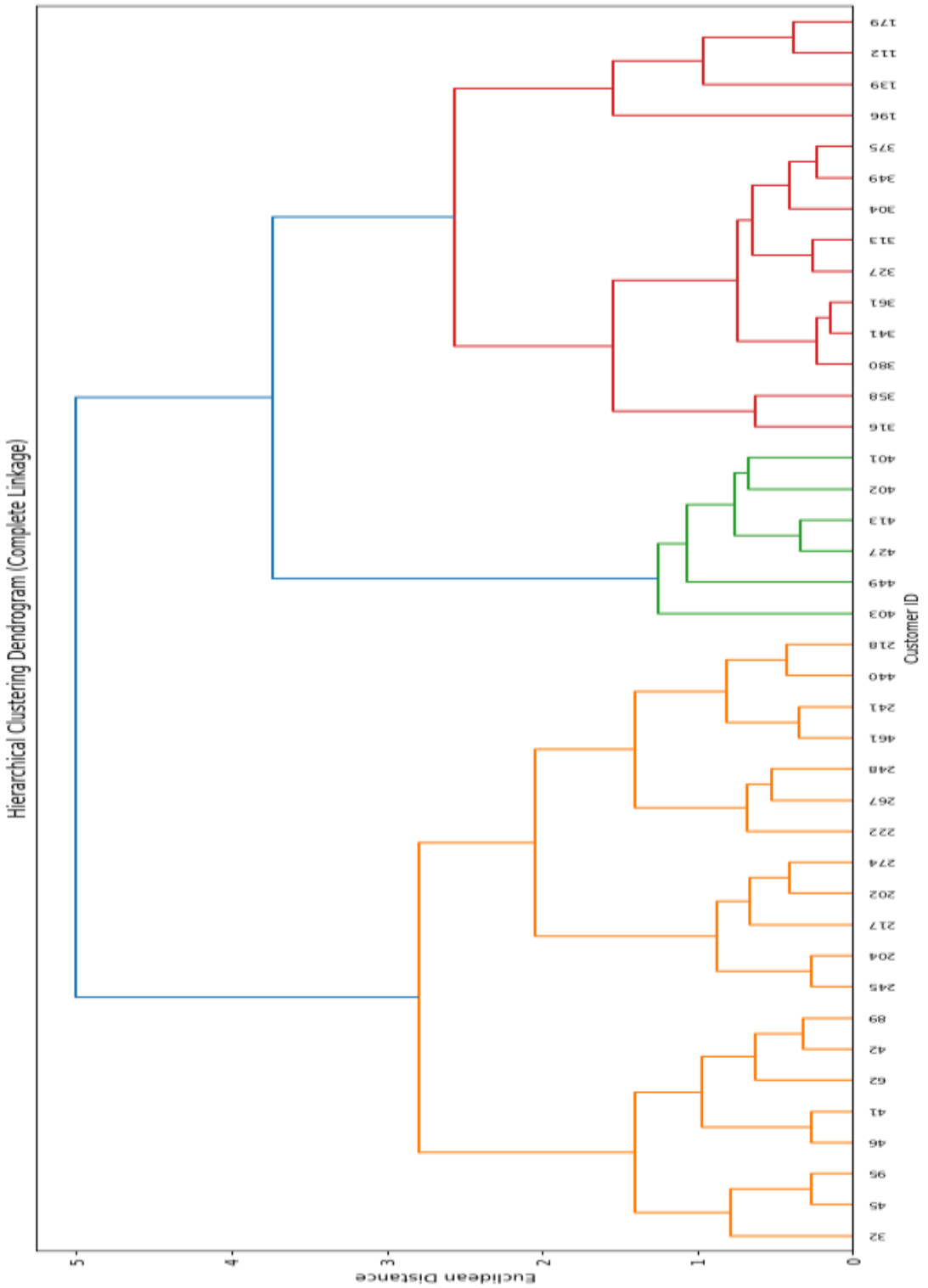
Total Manhattan distance cost: 575.192

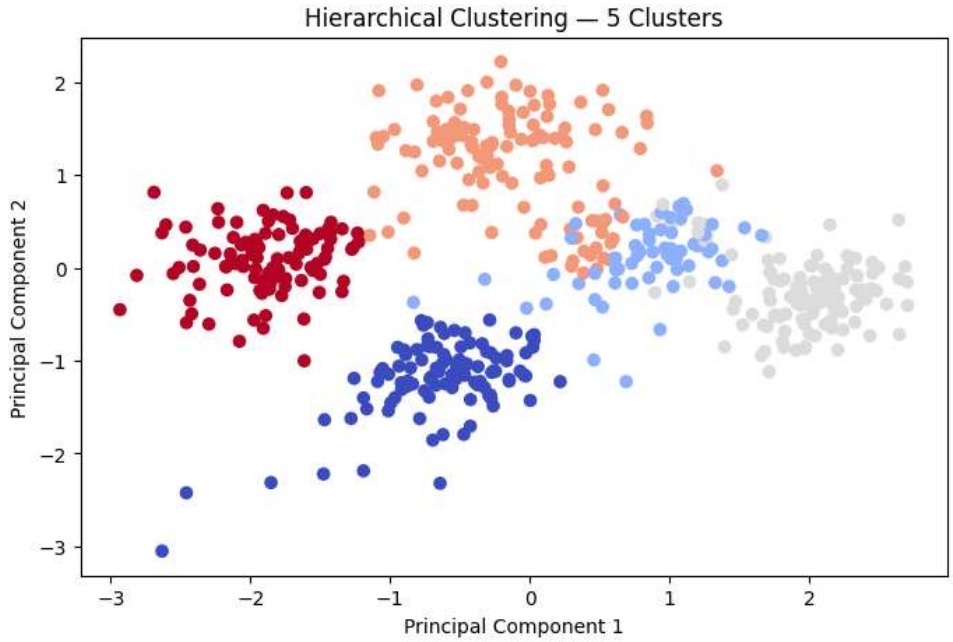
Hierarchical clustering:

```
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import linkage, dendrogram
# 1. Apply Hierarchical Clustering to all 500 records
hierarchical = AgglomerativeClustering(
    n_clusters=5,
    linkage="complete",
    metric="euclidean"
)
df["Hierarchical"] = hierarchical.fit_predict(X_scaled)
print("Cluster counts:")
print(df["Hierarchical"].value_counts().sort_index())
# 2. Draw a readable dendrogram using a 40-customer sample
sample_size = min(40, len(X_scaled))
rng = np.random.default_rng(42)
sample_indices = rng.choice(
    len(X_scaled), size=sample_size, replace=False
)
X_sample = X_scaled[sample_indices]
# Complete-linkage hierarchy
Z = linkage(X_sample, method="complete", metric="euclidean")
plt.figure(figsize=(16, 7))
dendrogram(
    Z,
    labels=df.iloc[sample_indices]["CustomerID"].astype(str).values,
    leaf_rotation=90,
    leaf_font_size=8
)
plt.title("Hierarchical Clustering Dendrogram (Complete Linkage)")
plt.xlabel("Customer ID")
plt.ylabel("Euclidean Distance")
plt.tight_layout()
plt.show()
# 3. Visualize cluster labels for all records
plt.figure(figsize=(8, 5))
plt.scatter(
    X_pca[:, 0], X_pca[:, 1],
    c=df["Hierarchical"], cmap="coolwarm", s=35
)
plt.title("Hierarchical Clustering — 5 Clusters")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.show()
```

```
Cluster counts:
Hierarchical
```

0 104
1 63
2 111
3 122
4 100
Name: count, dtype: int64

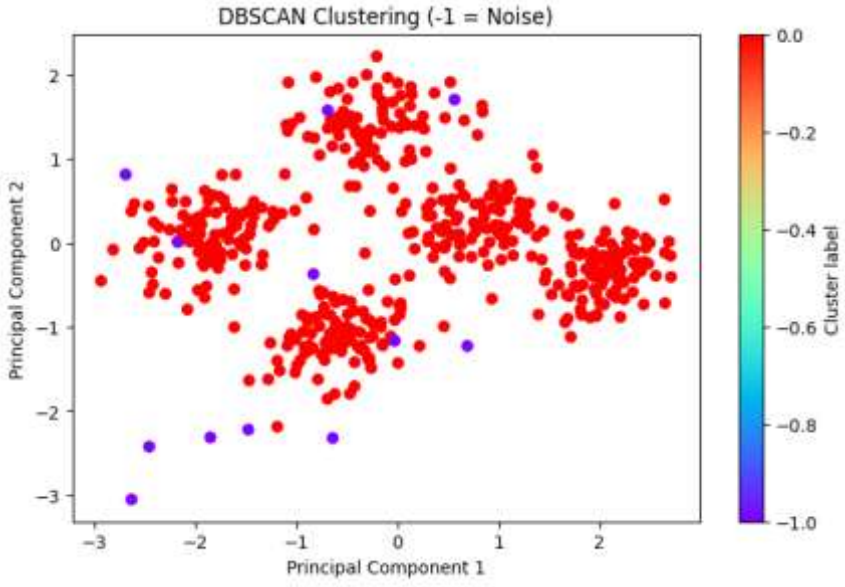



Hierarchical Clustering — 5 Clusters
Principal Component 2
Principal Component 1
2
1
0
−1
−2
−3
−3
−2
−1
0
1
2

DBSCAN clustering:

```
from sklearn.cluster import DBSCAN
dbscan = DBSCAN(
    eps=0.65,
    min_samples=8,
    metric="euclidean"
)
df["DBSCAN"] = dbscan.fit_predict(X_scaled)
print("Cluster counts:")
print(df["DBSCAN"].value_counts().sort_index())
# Noise points have label -1
noise_count = (df["DBSCAN"] == -1).sum()
print("\nNoise points:", noise_count)
# Plot clusters and noise
plt.figure(figsize=(8, 5))
plt.scatter(
    X_pca[:, 0], X_pca[:, 1],
    c=df["DBSCAN"], cmap="rainbow", s=35
)
plt.title("DBSCAN Clustering (-1 = Noise)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.colorbar(label="Cluster label")
plt.show()
```

```
Cluster counts:
DBSCAN
-1     12
 0    488
Name: count, dtype: int64
Noise points: 12
```



Comparison:

```
from sklearn.metrics import silhouette_score
cluster_columns = {
   "K-Means": "KMeans",
   "K-Medoids": "KMedoids",
   "Hierarchical": "Hierarchical",
   "DBSCAN": "DBSCAN"
}
comparison = []
for algorithm, column in cluster_columns.items():
   labels = df[column].to_numpy()
   # Exclude DBSCAN noise from the score
   mask = labels != -1
   valid_labels = labels[mask]
   number_of_clusters = len(np.unique(valid_labels))
   if number_of_clusters >= 2 and mask.sum() > number_of_clusters:
      score = round(
         silhouette_score(X_scaled[mask], valid_labels), 3
      )
   else:
      score = "N/A"
   comparison.append({
      "Algorithm": algorithm,
      "Clusters": number_of_clusters,
      "Noise Points": int(np.sum(labels == -1)),
      "Silhouette Score": score
   })
display(pd.DataFrame(comparison))
```

	Algorithm	Clusters	Noise Points	Silhouette Score
0	K-Means	5	0	0.505
1	K-Medoids	5	0	0.467
2	Hierarchical	5	0	0.43
3	DBSCAN	1	12	N/A

Final Results:

```
df.to_csv("customer_clustering_results.csv", index=False)
from google.colab import files
files.download("customer_clustering_results.csv")
```

	A	B	C	D	E	F	G	H
1	CustomerI	Age	AnnualInc	SpendingS	KMeans	KMedoids	Hierarchic	DBSCAN
2	1	23	16.7	87	0	3	2	0
3	2	26	9.4	68	0	3	2	0
4	3	23	22.5	80	0	3	2	0
5	4	19	32	87	0	3	2	0
6	5	22	34	84	0	3	2	0
7	6	19	28	71	0	3	2	0
8	7	26	24.6	78	0	3	2	0
9	8	19	34.8	79	0	3	2	0
10	9	20	22.2	85	0	3	2	0
11	10	23	28.3	84	0	3	2	0
12	11	31	21.7	75	0	3	2	0
13	12	19	29.9	90	0	3	2	0
14	13	22	18.3	73	0	3	2	0
15	14	25	30.9	85	0	3	2	0
16	15	19	26.9	81	0	3	2	0
17	16	23	32	82	0	3	2	0
18	17	25	25.5	83	0	3	2	0
19	18	25	13.3	77	0	3	2	0
20	19	20	19.9	78	0	3	2	0
21	20	28	18.1	89	0	3	2	0
22	21	18	22.3	81	0	3	2	0
23	22	24	30.7	87	0	3	2	0
24	23	21	21.3	88	0	3	2	0
25	24	21	14.8	70	0	3	2	0
26	25	18	29	81	0	3	2	0

customer_clustering_results

PHASE II - NEURAL NETWORK

```
import math
import matplotlib.pyplot as plt
from matplotlib.patches import Circle
def sigmoid(x):
    return 1 / (1 + math.exp(-x))
def draw_network(epoch, stage, weights, biases, y4=None, y5=None, y6=None):
    plt.close("all")
    fig, ax = plt.subplots(figsize=(9, 5.5))
    ax.set_xlim(0, 10)
    ax.set_ylim(0, 8)
    ax.axis("off")
    pos = {
        "x1": (1, 6.3),
        "x2": (1, 4.0),
        "x3": (1, 1.7),
        "y4": (5, 5.5),
        "y5": (5, 2.5),
        "y6": (9, 4.0)
    }
    edges = [
        ("x1", "y4", "w14"),
        ("x2", "y4", "w24"),
        ("x3", "y4", "w34"),
        ("x1", "y5", "w15"),
        ("x2", "y5", "w25"),
        ("x3", "y5", "w35"),
        ("y4", "y6", "w46"),
        ("y5", "y6", "w56")
    ]
    for start, end, weight_name in edges:
        x_start, y_start = pos[start]
        x_end, y_end = pos[end]
        ax.annotate(
            "",
            xy=(x_end - 0.38, y_end),
            xytext=(x_start + 0.38, y_start),
            arrowprops=dict(
                arrowstyle="->",
                linewidth=1
            )
        )
        mid_x = (x_start + x_end) / 2
        mid_y = (y_start + y_end) / 2
        ax.text(
            mid_x,
```

```
        mid_y + 0.16,
        f"{weight_name}={weights[weight_name]:.3f}",
        fontsize=7,
        ha="center"
    )
node_text = {
    "x1": "x1\n1",
    "x2": "x2\n0",
    "x3": "x3\n1",
    "y4": f"y4\n{y4:.4f}" if y4 is not None else "y4",
    "y5": f"y5\n{y5:.4f}" if y5 is not None else "y5",
    "y6": f"y6\n{y6:.4f}" if y6 is not None else "y6"
}
for node, (x, y) in pos.items():
    circle = Circle(
        (x, y),
        0.4,
        fill=False,
        linewidth=1.5
    )
    ax.add_patch(circle)
    ax.text(
        x,
        y,
        node_text[node],
        ha="center",
        va="center",
        fontsize=9
    )
ax.annotate(
    "",
    xy=(5, 5.9),
    xytext=(5, 6.8),
    arrowprops=dict(
        arrowstyle="->",
        linewidth=1
    )
)
ax.text(
    5,
    7.05,
    f"b4={biases['b4']:.3f}",
    ha="center",
    fontsize=8
)
ax.annotate(
```

```
    "",
    xy=(5, 2.1),
    xytext=(5, 1.0),
    arrowprops=dict(
        arrowstyle="->",
        linewidth=1
    )
)
ax.text(
    5,
    0.7,
    f"b5={biases['b5']:.3f}",
    ha="center",
    fontsize=8
)
ax.annotate(
    "",
    xy=(9, 4.4),
    xytext=(9, 5.7),
    arrowprops=dict(
        arrowstyle="->",
        linewidth=1
    )
)
ax.text(
    9,
    6.0,
    f"b6={biases['b6']:.3f}",
    ha="center",
    fontsize=8
)
ax.text(
    1,
    7.55,
    "INPUT",
    fontsize=11,
    ha="center"
)
ax.text(
    5,
    7.55,
    "HIDDEN",
    fontsize=11,
    ha="center"
)
ax.text(
```

```
        9,
        7.55,
        "OUTPUT",
        fontsize=11,
        ha="center"
    )
    ax.set_title(
        f"Epoch {epoch} - {stage}",
        fontsize=12
    )
    plt.tight_layout()
    plt.show(block=False)
    plt.pause(1)
    plt.close()
def get_weights():
    return {
        "w14": w14,
        "w24": w24,
        "w34": w34,
        "w15": w15,
        "w25": w25,
        "w35": w35,
        "w46": w46,
        "w56": w56
    }
def get_biases():
    return {
        "b4": b4,
        "b5": b5,
        "b6": b6
    }
x1 = 1
x2 = 0
x3 = 1
target = 1
learning_rate = 0.9
max_epochs = 10
w14 = 0.2
w24 = 0.4
w34 = -0.5
w15 = -0.3
w25 = 0.1
w35 = 0.2
w46 = -0.3
w56 = -0.2
b4 = -0.4
```

```
b5 = 0.2
b6 = 0.1
errors = []
print()
print("==================================================")
print(" INITIAL NETWORK")
print("==================================================")
draw_network(
    0,
    "Initial Weights",
    get_weights(),
    get_biases()
)
for epoch in range(1, max_epochs + 1):
    print()
    print()
    print("==================================================")
    print(f" EPOCH {epoch}")
    print("==================================================")
    print()
    print("*************** FORWARD PROPAGATION ***************")
    net4 = (
        x1 * w14 +
        x2 * w24 +
        x3 * w34 +
        b4
    )
    print()
    print("Step 1: Calculate net4")
    print(
        f"net4 = ({x1} × {w14:.6f}) + "
        f"({x2} × {w24:.6f}) + "
        f"({x3} × {w34:.6f}) + "
        f"{b4:.6f}"
    )
    print(f"net4 = {net4:.6f}")
    y4 = sigmoid(net4)
    print()
    print("Step 2: Calculate y4")
    print(f"y4 = {y4:.6f}")
    net5 = (
        x1 * w15 +
        x2 * w25 +
        x3 * w35 +
        b5
    )
```

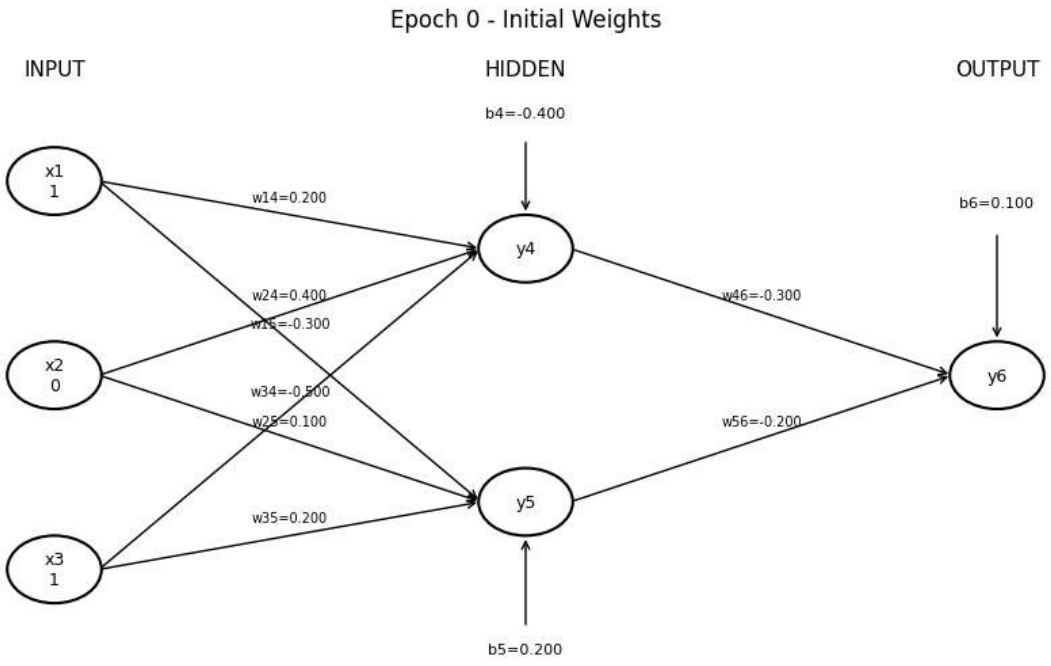
```
print()
print("Step 3: Calculate net5")
print(
    f"net5 = ({x1} × {w15:.6f}) + "
    f"({x2} × {w25:.6f}) + "
    f"({x3} × {w35:.6f}) + "
    f"{b5:.6f}"
)
print(f"net5 = {net5:.6f}")
y5 = sigmoid(net5)
print()
print("Step 4: Calculate y5")
print(f"y5 = {y5:.6f}")
net6 = (
    y4 * w46 +
    y5 * w56 +
    b6
)
print()
print("Step 5: Calculate net6")
print(
    f"net6 = ({y4:.6f} × {w46:.6f}) + "
    f"({y5:.6f} × {w56:.6f}) + "
    f"{b6:.6f}"
)
print(f"net6 = {net6:.6f}")
y6 = sigmoid(net6)
print()
print("Step 6: Calculate y6")
print(f"y6 = {y6:.6f}")
error = abs(target - y6)
errors.append(error)
print()
print("Step 7: Calculate Error")
print(
    f"Error = |{target:.6f} - {y6:.6f}|"
)
print(
    f"Error = {error:.6f}"
)
print()
print("Step 8: Output")
print(f"Expected output = {target:.6f}")
print(f"Predicted output = {y6:.6f}")
print()
print("*************** BACKWARD PROPAGATION ***************")
```

```
delta6 = (
    (target - y6)
    * y6
    * (1 - y6)
)
print()
print("Step 9: Calculate delta6")
print(f"delta6 = {delta6:.6f}")
delta4 = (
    y4 *
    (1 - y4) *
    delta6 *
    w46
)
print()
print("Step 10: Calculate delta4")
print(f"delta4 = {delta4:.6f}")
delta5 = (
    y5 *
    (1 - y5) *
    delta6 *
    w56
)
print()
print("Step 11: Calculate delta5")
print(f"delta5 = {delta5:.6f}")
print()
print("*************** DELTA WEIGHTS ***************")
delta_w46 = learning_rate * delta6 * y4
delta_w56 = learning_rate * delta6 * y5
delta_b6 = learning_rate * delta6
delta_w14 = learning_rate * delta4 * x1
delta_w24 = learning_rate * delta4 * x2
delta_w34 = learning_rate * delta4 * x3
delta_b4 = learning_rate * delta4
delta_w15 = learning_rate * delta5 * x1
delta_w25 = learning_rate * delta5 * x2
delta_w35 = learning_rate * delta5 * x3
delta_b5 = learning_rate * delta5
print()
print(f"Δw46 = {delta_w46:.6f}")
print(f"Δw56 = {delta_w56:.6f}")
print(f"Δb6 = {delta_b6:.6f}")
print()
print(f"Δw14 = {delta_w14:.6f}")
print(f"Δw24 = {delta_w24:.6f}")
```

```
print(f"Δw34 = {delta_w34:.6f}")
print(f"Δb4 = {delta_b4:.6f}")
print()
print(f"Δw15 = {delta_w15:.6f}")
print(f"Δw25 = {delta_w25:.6f}")
print(f"Δw35 = {delta_w35:.6f}")
print(f"Δb5 = {delta_b5:.6f}")
print()
print("********** UPDATING WEIGHTS **********")
w46 = w46 + delta_w46
w56 = w56 + delta_w56
w14 = w14 + delta_w14
w24 = w24 + delta_w24
w34 = w34 + delta_w34
w15 = w15 + delta_w15
w25 = w25 + delta_w25
w35 = w35 + delta_w35
b4 = b4 + delta_b4
b5 = b5 + delta_b5
b6 = b6 + delta_b6
print()
print("Updated weights:")
print(f"w14 = {w14:.6f}")
print(f"w24 = {w24:.6f}")
print(f"w34 = {w34:.6f}")
print(f"w15 = {w15:.6f}")
print(f"w25 = {w25:.6f}")
print(f"w35 = {w35:.6f}")
print(f"w46 = {w46:.6f}")
print(f"w56 = {w56:.6f}")
print()
print("Updated biases:")
print(f"b4 = {b4:.6f}")
print(f"b5 = {b5:.6f}")
print(f"b6 = {b6:.6f}")
print()
print("Generating updated graph...")
draw_network(
    epoch,
    "After Weight Update",
    get_weights(),
    get_biases(),
    y4,
    y5,
    y6
)
```

```
print()
print()
print("=================================================")
print(" TRAINING COMPLETED")
print("=================================================")
print(f"Total epochs completed : {max_epochs}")
print()
print(f"Target value : {target:.6f}")
print(f"Final predicted output : {y6:.6f}")
print(f"Final error : {error:.6f}")
print()
print("=================================================")
print(" ERROR FOR ALL 10 EPOCHS")
print("=================================================")
for i, err in enumerate(errors, start=1):
    print(
        f"Epoch {i:2d} : Error = {err:.6f}"
    )
print()
print("=================================================")
print(" FINAL COMPARISON")
print("=================================================")
print(f"Expected output : {target:.6f}")
print(f"Predicted output: {y6:.6f}")
print(f"Difference      : {abs(target - y6):.6f}")
```

```
==================================================
 INITIAL NETWORK
==================================================
```



==========
EPOCH 1
==========

********** FORWARD PROPAGATION **********

Step 1: Calculate net4
net4 = (1 × 0.200000) + (0 × 0.400000) + (1 × -0.500000) + -0.400000
net4 = -0.700000

Step 2: Calculate y4
y4 = 0.331812

Step 3: Calculate net5
net5 = (1 × -0.300000) + (0 × 0.100000) + (1 × 0.200000) + 0.200000
net5 = 0.100000

Step 4: Calculate y5
y5 = 0.524979

Step 5: Calculate net6
net6 = (0.331812 × -0.300000) + (0.524979 × -0.200000) + 0.100000
net6 = -0.104540

Step 6: Calculate y6
y6 = 0.473889

Step 7: Calculate Error
Error = |1.000000 - 0.473889|
Error = 0.526111

Step 8: Output
Expected output = 1.000000
Predicted output = 0.473889

********** BACKWARD PROPAGATION **********

Step 9: Calculate delta6
delta6 = 0.131169

Step 10: Calculate delta4
delta4 = -0.008725

Step 11: Calculate delta5
delta5 = -0.006542

********** DELTA WEIGHTS **********

Δw46 = 0.039171
Δw56 = 0.061975
Δb6 = 0.118052

Δw14 = -0.007852
Δw24 = -0.000000
Δw34 = -0.007852
Δb4 = -0.007852

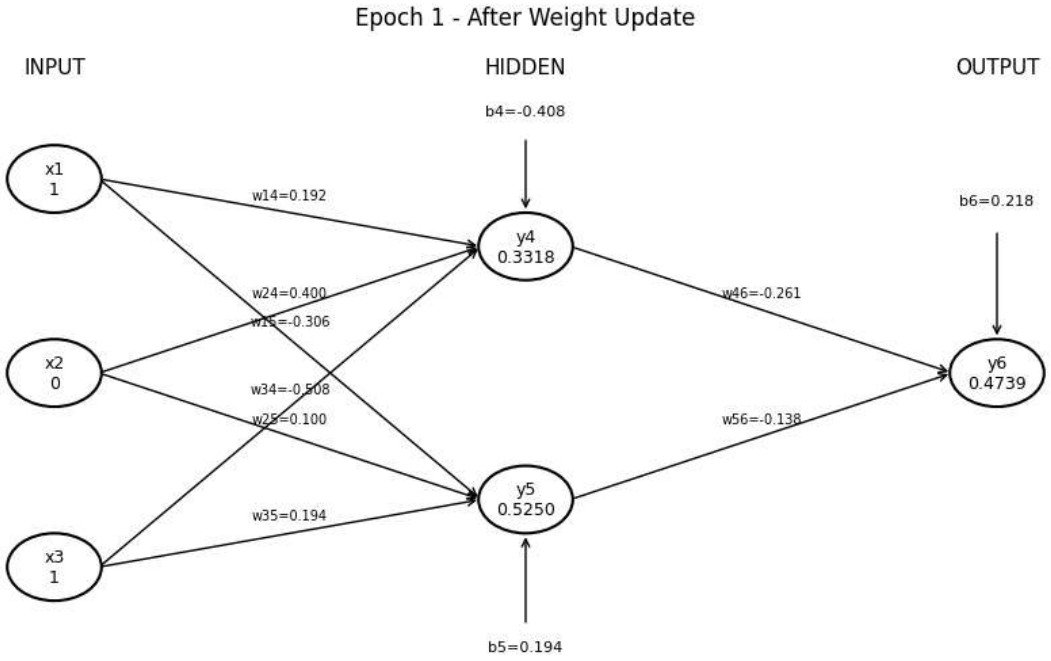
Δw15 = -0.005888
Δw25 = -0.000000
Δw35 = -0.005888
Δb5 = -0.005888

********** UPDATING WEIGHTS **********

Updated weights:
w14 = 0.192148
w24 = 0.400000
w34 = -0.507852
w15 = -0.305888
w25 = 0.100000
w35 = 0.194112
w46 = -0.260829
w56 = -0.138025

Updated biases:
b4 = -0.407852
b5 = 0.194112
b6 = 0.218052

Generating updated graph...



==========

EPOCH 2

==========

********** FORWARD PROPAGATION **********

Step 1: Calculate net4
net4 = (1 × 0.192148) + (0 × 0.400000) + (1 × -0.507852) + -0.407852
net4 = -0.723556

Step 2: Calculate y4
y4 = 0.326610

Step 3: Calculate net5
net5 = (1 × -0.305888) + (0 × 0.100000) + (1 × 0.194112) + 0.194112
net5 = 0.082336

Step 4: Calculate y5
y5 = 0.520572

Step 5: Calculate net6
net6 = (0.326610 × -0.260829) + (0.520572 × -0.138025) + 0.218052
net6 = 0.061011

Step 6: Calculate y6
y6 = 0.515248

Step 7: Calculate Error
Error = |1.000000 - 0.515248|
Error = 0.484752

Step 8: Output
Expected output = 1.000000
Predicted output = 0.515248

********** BACKWARD PROPAGATION **********

Step 9: Calculate delta6
delta6 = 0.121075

Step 10: Calculate delta4
delta4 = -0.006946

Step 11: Calculate delta5
delta5 = -0.004171

********** DELTA WEIGHTS **********

Δw46 = 0.035590
Δw56 = 0.056726
Δb6 = 0.108968

Δw14 = -0.006251
Δw24 = -0.000000
Δw34 = -0.006251
Δb4 = -0.006251

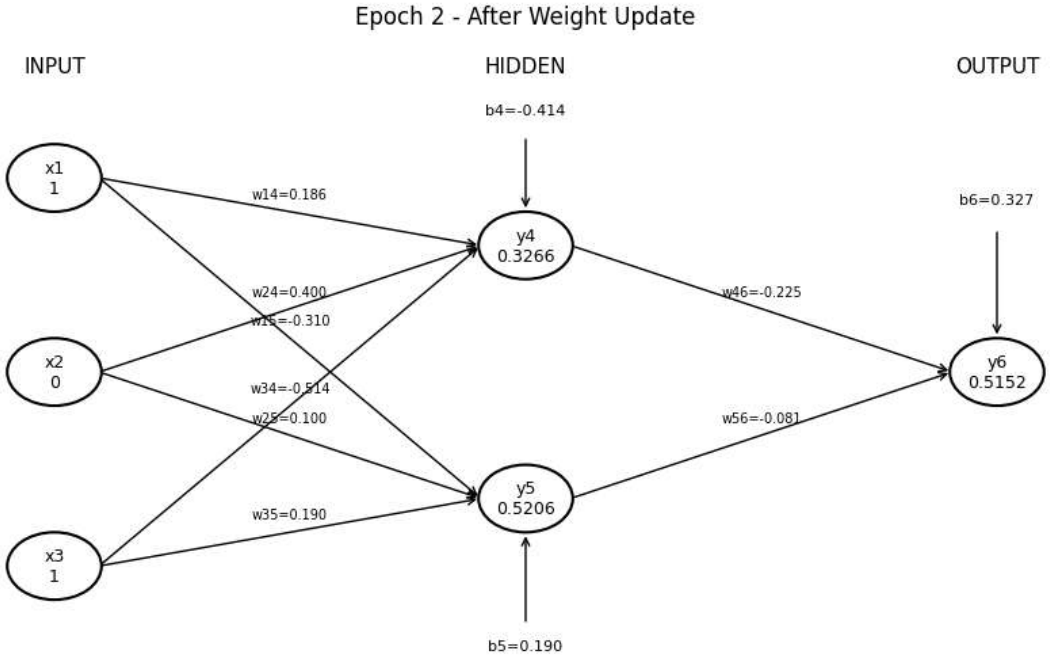
Δw15 = -0.003754
Δw25 = -0.000000
Δw35 = -0.003754
Δb5 = -0.003754

********** UPDATING WEIGHTS **********

Updated weights:
w14 = 0.185897
w24 = 0.400000
w34 = -0.514103
w15 = -0.309642
w25 = 0.100000
w35 = 0.190358
w46 = -0.225239
w56 = -0.081299

Updated biases:
b4 = -0.414103
b5 = 0.190358
b6 = 0.327020

Generating updated graph...



==========

EPOCH 3

==========

********** FORWARD PROPAGATION **********

Step 1: Calculate net4
net4 = (1 × 0.185897) + (0 × 0.400000) + (1 × -0.514103) + -0.414103
net4 = -0.742309

Step 2: Calculate y4
y4 = 0.322499

Step 3: Calculate net5
net5 = (1 × -0.309642) + (0 × 0.100000) + (1 × 0.190358) + 0.190358
net5 = 0.071075

Step 4: Calculate y5
y5 = 0.517761

Step 5: Calculate net6
net6 = (0.322499 × -0.225239) + (0.517761 × -0.081299) + 0.327020
net6 = 0.212287

Step 6: Calculate y6
y6 = 0.552873

Step 7: Calculate Error
Error = |1.000000 - 0.552873|
Error = 0.447127

Step 8: Output
Expected output = 1.000000
Predicted output = 0.552873

********** BACKWARD PROPAGATION **********

Step 9: Calculate delta6
delta6 = 0.110532

Step 10: Calculate delta4
delta4 = -0.005440

Step 11: Calculate delta5
delta5 = -0.002244

********** DELTA WEIGHTS **********

Δw46 = 0.032082
Δw56 = 0.051506
Δb6 = 0.099479

Δw14 = -0.004896
Δw24 = -0.000000
Δw34 = -0.004896
Δb4 = -0.004896

Δw15 = -0.002019
Δw25 = -0.000000
Δw35 = -0.002019
Δb5 = -0.002019

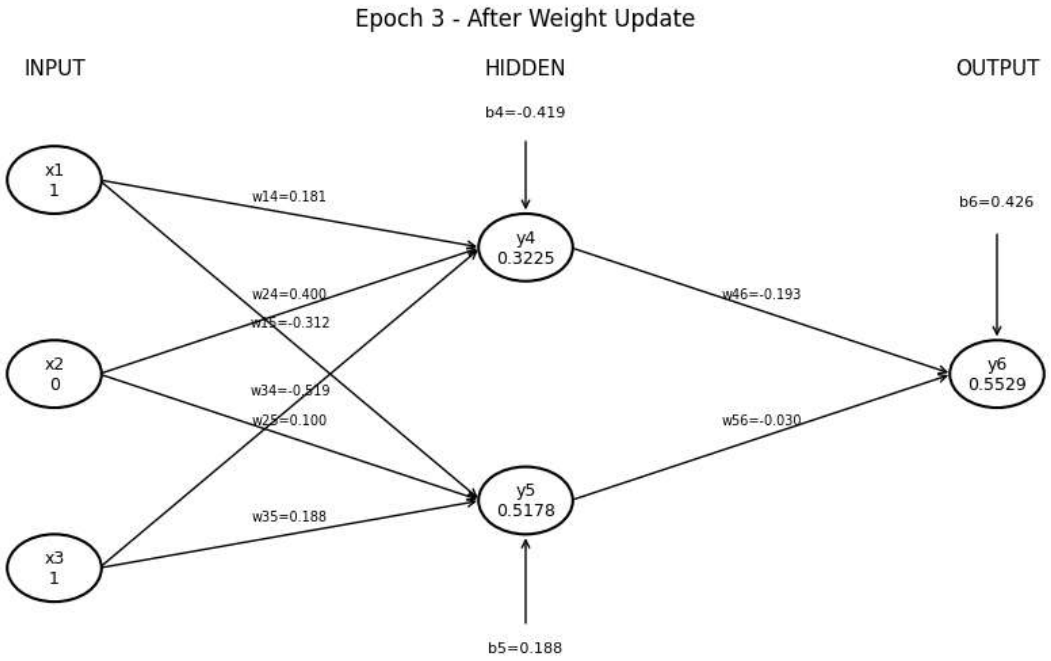
********** UPDATING WEIGHTS **********

Updated weights:
w14 = 0.181001

w24 = 0.400000
w34 = -0.518999
w15 = -0.311661
w25 = 0.100000
w35 = 0.188339
w46 = -0.193157
w56 = -0.029793

Updated biases:
b4 = -0.418999
b5 = 0.188339
b6 = 0.426498

Generating updated graph...



==========

EPOCH 4

==========

********** FORWARD PROPAGATION **********

Step 1: Calculate net4
net4 = (1 × 0.181001) + (0 × 0.400000) + (1 × -0.518999) + -0.418999
net4 = -0.756996

Step 2: Calculate y4
y4 = 0.319299

Step 3: Calculate net5
net5 = (1 × -0.311661) + (0 × 0.100000) + (1 × 0.188339) + 0.188339
net5 = 0.065017

Step 4: Calculate y5
y5 = 0.516249

Step 5: Calculate net6
net6 = (0.319299 × -0.193157) + (0.516249 × -0.029793) + 0.426498
net6 = 0.349443

Step 6: Calculate y6
y6 = 0.586482

Step 7: Calculate Error
Error = |1.000000 - 0.586482|
Error = 0.413518

Step 8: Output
Expected output = 1.000000
Predicted output = 0.586482

********** BACKWARD PROPAGATION **********

Step 9: Calculate delta6
delta6 = 0.100287

Step 10: Calculate delta4
delta4 = -0.004210

Step 11: Calculate delta5
delta5 = -0.000746

********** DELTA WEIGHTS **********

Δw46 = 0.028819
Δw56 = 0.046596
Δb6 = 0.090258

Δw14 = -0.003789
Δw24 = -0.000000
Δw34 = -0.003789
Δb4 = -0.003789

Δw15 = -0.000672
Δw25 = -0.000000

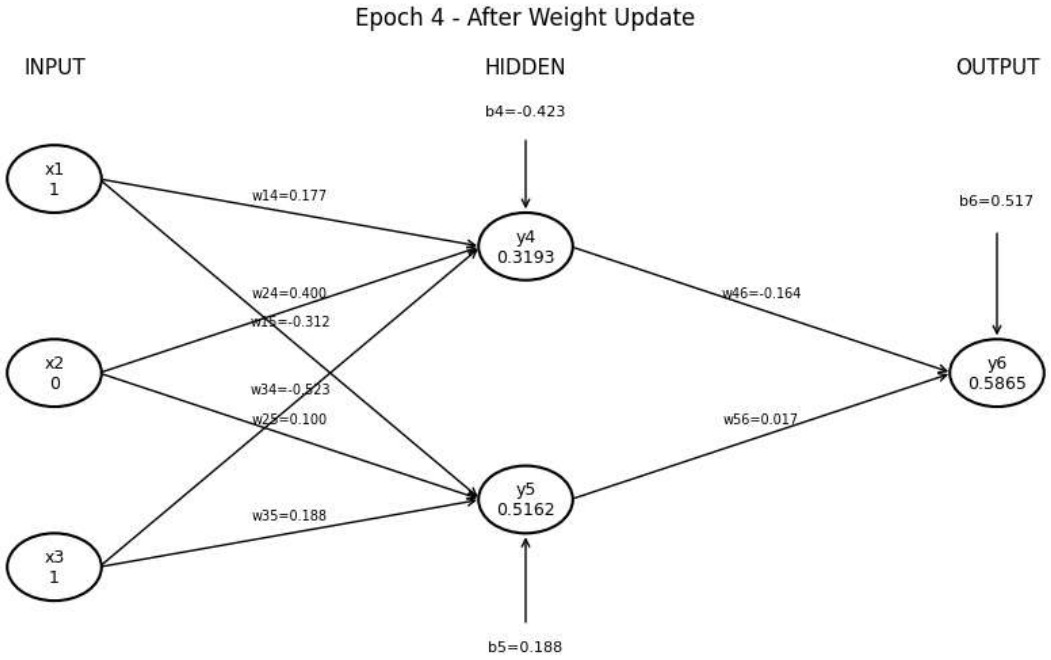
Δw35 = -0.000672
Δb5 = -0.000672

********** UPDATING WEIGHTS **********

Updated weights:
w14 = 0.177212
w24 = 0.400000
w34 = -0.522788
w15 = -0.312332
w25 = 0.100000
w35 = 0.187668
w46 = -0.164338
w56 = 0.016802

Updated biases:
b4 = -0.422788
b5 = 0.187668
b6 = 0.516756

Generating updated graph...



==========

EPOCH 5

==========

********** FORWARD PROPAGATION **********

Step 1: Calculate net4
net4 = (1 × 0.177212) + (0 × 0.400000) + (1 × -0.522788) + -0.422788
net4 = -0.768364

Step 2: Calculate y4
y4 = 0.316833

Step 3: Calculate net5
net5 = (1 × -0.312332) + (0 × 0.100000) + (1 × 0.187668) + 0.187668
net5 = 0.063003

Step 4: Calculate y5
y5 = 0.515745

Step 5: Calculate net6
net6 = (0.316833 × -0.164338) + (0.515745 × 0.016802) + 0.516756
net6 = 0.473354

Step 6: Calculate y6
y6 = 0.616177

Step 7: Calculate Error
Error = |1.000000 - 0.616177|
Error = 0.383823

Step 8: Output
Expected output = 1.000000
Predicted output = 0.616177

********** BACKWARD PROPAGATION **********

Step 9: Calculate delta6
delta6 = 0.090775

Step 10: Calculate delta4
delta4 = -0.003229

Step 11: Calculate delta5
delta5 = 0.000381

********** DELTA WEIGHTS **********

Δw46 = 0.025885
Δw56 = 0.042135
Δb6 = 0.081698

Δw14 = -0.002906
Δw24 = -0.000000
Δw34 = -0.002906
Δb4 = -0.002906

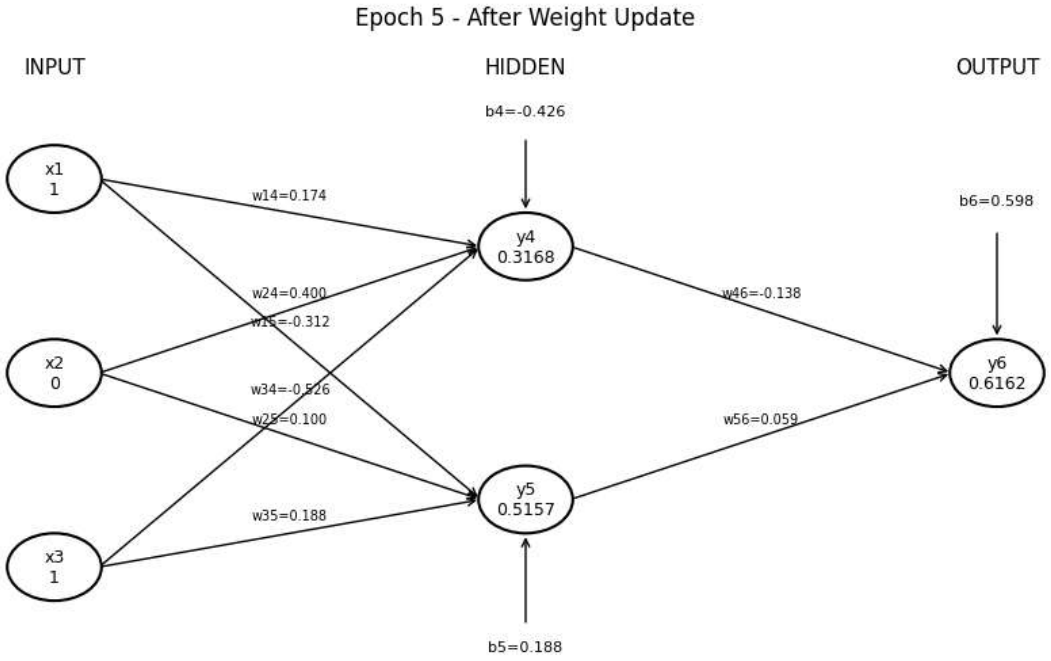
Δw15 = 0.000343
Δw25 = 0.000000
Δw35 = 0.000343
Δb5 = 0.000343

********** UPDATING WEIGHTS **********

Updated weights:
w14 = 0.174306
w24 = 0.400000
w34 = -0.525694
w15 = -0.311990
w25 = 0.100000
w35 = 0.188010
w46 = -0.138453
w56 = 0.058937

Updated biases:
b4 = -0.425694
b5 = 0.188010
b6 = 0.598454

Generating updated graph...



==========
EPOCH 6
==========

********** FORWARD PROPAGATION **********

Step 1: Calculate net4
net4 = (1 × 0.174306) + (0 × 0.400000) + (1 × -0.525694) + -0.425694
net4 = -0.777082

Step 2: Calculate y4
y4 = 0.314949

Step 3: Calculate net5
net5 = (1 × -0.311990) + (0 × 0.100000) + (1 × 0.188010) + 0.188010
net5 = 0.064031

Step 4: Calculate y5
y5 = 0.516002

Step 5: Calculate net6
net6 = (0.314949 × -0.138453) + (0.516002 × 0.058937) + 0.598454
net6 = 0.585260

Step 6: Calculate y6
y6 = 0.642277

Step 7: Calculate Error
Error = |1.000000 - 0.642277|
Error = 0.357723

Step 8: Output
Expected output = 1.000000
Predicted output = 0.642277

********** BACKWARD PROPAGATION **********

Step 9: Calculate delta6
delta6 = 0.082190

Step 10: Calculate delta4
delta4 = -0.002455

Step 11: Calculate delta5
delta5 = 0.001210

********** DELTA WEIGHTS **********

Δw46 = 0.023297
Δw56 = 0.038169
Δb6 = 0.073971

Δw14 = -0.002210
Δw24 = -0.000000
Δw34 = -0.002210
Δb4 = -0.002210

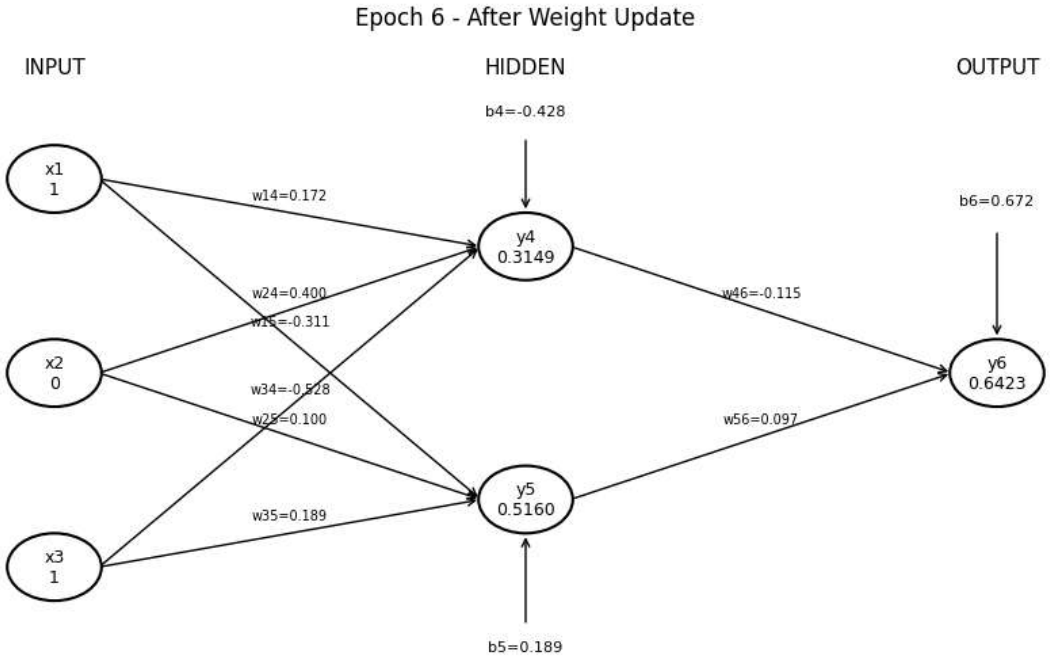
Δw15 = 0.001089
Δw25 = 0.000000
Δw35 = 0.001089
Δb5 = 0.001089

********** UPDATING WEIGHTS **********

Updated weights:
w14 = 0.172096
w24 = 0.400000
w34 = -0.527904
w15 = -0.310901
w25 = 0.100000
w35 = 0.189099
w46 = -0.115156
w56 = 0.097106

Updated biases:
b4 = -0.427904
b5 = 0.189099
b6 = 0.672425

Generating updated graph...



==========

EPOCH 7

==========

********** FORWARD PROPAGATION **********

Step 1: Calculate net4
net4 = (1 × 0.172096) + (0 × 0.400000) + (1 × -0.527904) + -0.427904
net4 = -0.783711

Step 2: Calculate y4
y4 = 0.313521

Step 3: Calculate net5
net5 = (1 × -0.310901) + (0 × 0.100000) + (1 × 0.189099) + 0.189099
net5 = 0.067297

Step 4: Calculate y5
y5 = 0.516818

Step 5: Calculate net6
net6 = (0.313521 × -0.115156) + (0.516818 × 0.097106) + 0.672425
net6 = 0.686507

Step 6: Calculate y6
y6 = 0.665189

Step 7: Calculate Error
Error = |1.000000 - 0.665189|
Error = 0.334811

Step 8: Output
Expected output = 1.000000
Predicted output = 0.665189

********** BACKWARD PROPAGATION **********

Step 9: Calculate delta6
delta6 = 0.074566

Step 10: Calculate delta4
delta4 = -0.001848

Step 11: Calculate delta5
delta5 = 0.001808

********** DELTA WEIGHTS **********

Δw46 = 0.021040
Δw56 = 0.034684
Δb6 = 0.067110

Δw14 = -0.001663
Δw24 = -0.000000
Δw34 = -0.001663
Δb4 = -0.001663

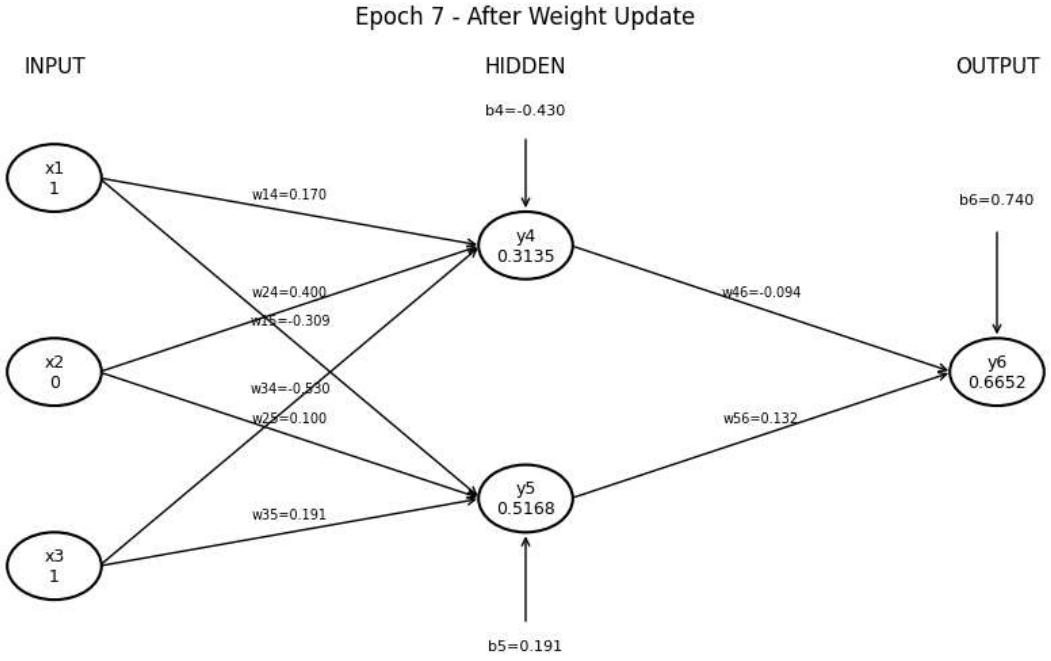
Δw15 = 0.001627
Δw25 = 0.000000
Δw35 = 0.001627
Δb5 = 0.001627

********** UPDATING WEIGHTS **********

Updated weights:
w14 = 0.170433
w24 = 0.400000
w34 = -0.529567
w15 = -0.309273
w25 = 0.100000
w35 = 0.190727
w46 = -0.094116
w56 = 0.131790

Updated biases:
b4 = -0.429567
b5 = 0.190727
b6 = 0.739534

Generating updated graph...



==========

EPOCH 8

==========

********** FORWARD PROPAGATION **********

Step 1: Calculate net4
net4 = (1 × 0.170433) + (0 × 0.400000) + (1 × -0.529567) + -0.429567
net4 = -0.788701

Step 2: Calculate y4
y4 = 0.312448

Step 3: Calculate net5
net5 = (1 × -0.309273) + (0 × 0.100000) + (1 × 0.190727) + 0.190727
net5 = 0.072180

Step 4: Calculate y5
y5 = 0.518037

Step 5: Calculate net6
net6 = (0.312448 × -0.094116) + (0.518037 × 0.131790) + 0.739534
net6 = 0.778400

Step 6: Calculate y6
y6 = 0.685335

Step 7: Calculate Error
Error = |1.000000 - 0.685335|
Error = 0.314665

Step 8: Output
Expected output = 1.000000
Predicted output = 0.685335

********** BACKWARD PROPAGATION **********

Step 9: Calculate delta6
delta6 = 0.067858

Step 10: Calculate delta4
delta4 = -0.001372

Step 11: Calculate delta5
delta5 = 0.002233

********** DELTA WEIGHTS **********

Δw46 = 0.019082
Δw56 = 0.031638
Δb6 = 0.061072

Δw14 = -0.001235
Δw24 = -0.000000
Δw34 = -0.001235
Δb4 = -0.001235

Δw15 = 0.002010
Δw25 = 0.000000
Δw35 = 0.002010
Δb5 = 0.002010

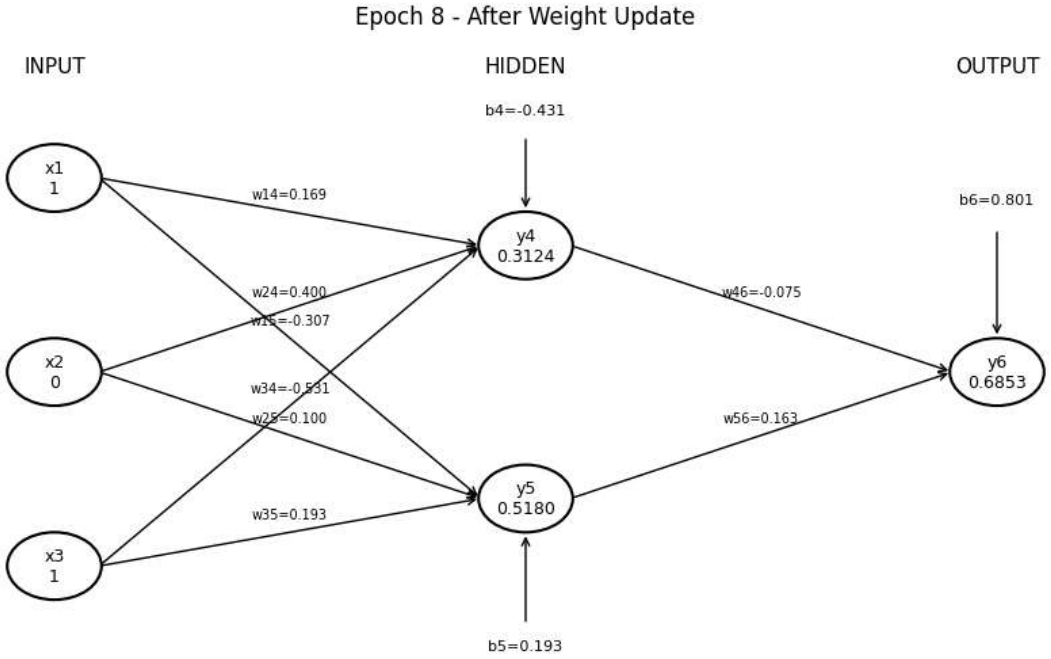
********** UPDATING WEIGHTS **********

Updated weights:
w14 = 0.169198

w24 = 0.400000
w34 = -0.530802
w15 = -0.307264
w25 = 0.100000
w35 = 0.192736
w46 = -0.075034
w56 = 0.163427

Updated biases:
b4 = -0.430802
b5 = 0.192736
b6 = 0.800606

Generating updated graph...



==========

EPOCH 9

==========

********** FORWARD PROPAGATION **********

Step 1: Calculate net4
net4 = (1 × 0.169198) + (0 × 0.400000) + (1 × -0.530802) + -0.430802
net4 = -0.792405

Step 2: Calculate y4
y4 = 0.311652

Step 3: Calculate net5
net5 = (1 × -0.307264) + (0 × 0.100000) + (1 × 0.192736) + 0.192736
net5 = 0.078208

Step 4: Calculate y5
y5 = 0.519542

Step 5: Calculate net6
net6 = (0.311652 × -0.075034) + (0.519542 × 0.163427) + 0.800606
net6 = 0.862129

Step 6: Calculate y6
y6 = 0.703105

Step 7: Calculate Error
Error = |1.000000 - 0.703105|
Error = 0.296895

Step 8: Output
Expected output = 1.000000
Predicted output = 0.703105

********** BACKWARD PROPAGATION **********

Step 9: Calculate delta6
delta6 = 0.061976

Step 10: Calculate delta4
delta4 = -0.000998

Step 11: Calculate delta5
delta5 = 0.002528

********** DELTA WEIGHTS **********

Δw46 = 0.017384
Δw56 = 0.028979
Δb6 = 0.055779

Δw14 = -0.000898
Δw24 = -0.000000
Δw34 = -0.000898
Δb4 = -0.000898

Δw15 = 0.002275
Δw25 = 0.000000

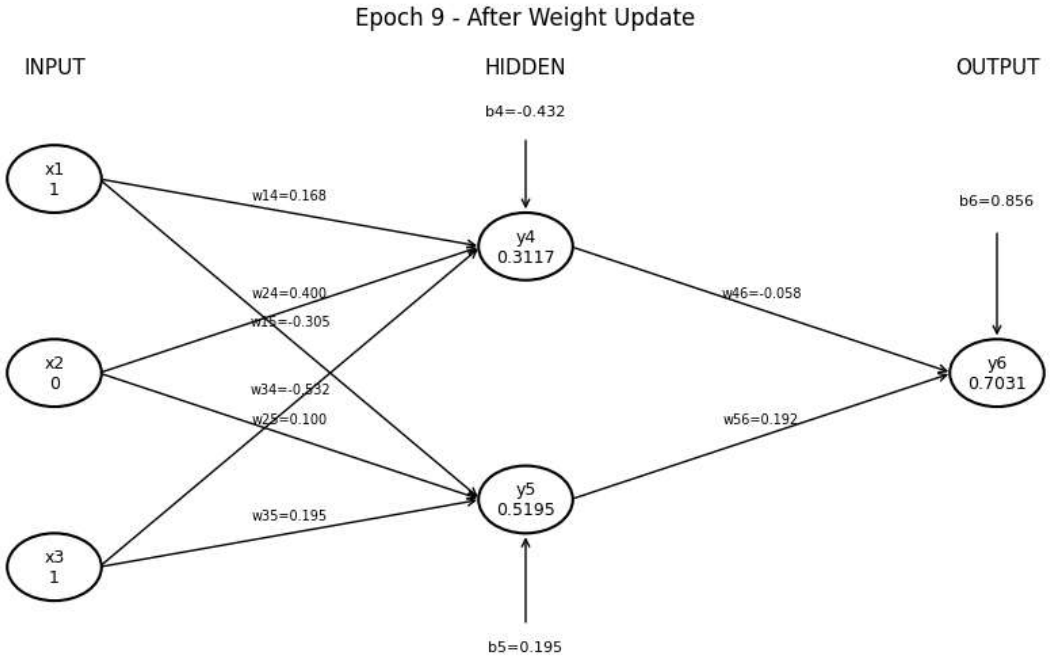
Δw35 = 0.002275
Δb5 = 0.002275

********** UPDATING WEIGHTS **********

Updated weights:
w14 = 0.168300
w24 = 0.400000
w34 = -0.531700
w15 = -0.304988
w25 = 0.100000
w35 = 0.195012
w46 = -0.057651
w56 = 0.192407

Updated biases:
b4 = -0.431700
b5 = 0.195012
b6 = 0.856385

Generating updated graph...



==========

EPOCH 10

==========

********** FORWARD PROPAGATION **********

Step 1: Calculate net4
net4 = (1 × 0.168300) + (0 × 0.400000) + (1 × -0.531700) + -0.431700
net4 = -0.795099

Step 2: Calculate y4
y4 = 0.311075

Step 3: Calculate net5
net5 = (1 × -0.304988) + (0 × 0.100000) + (1 × 0.195012) + 0.195012
net5 = 0.085035

Step 4: Calculate y5
y5 = 0.521246

Step 5: Calculate net6
net6 = (0.311075 × -0.057651) + (0.521246 × 0.192407) + 0.856385
net6 = 0.938743

Step 6: Calculate y6
y6 = 0.718846

Step 7: Calculate Error
Error = |1.000000 - 0.718846|
Error = 0.281154

Step 8: Output
Expected output = 1.000000
Predicted output = 0.718846

********** BACKWARD PROPAGATION **********

Step 9: Calculate delta6
delta6 = 0.056823

Step 10: Calculate delta4
delta4 = -0.000702

Step 11: Calculate delta5
delta5 = 0.002728

********** DELTA WEIGHTS **********

Δw46 = 0.015909
Δw56 = 0.026657
Δb6 = 0.051141

Δw14 = -0.000632
Δw24 = -0.000000
Δw34 = -0.000632
Δb4 = -0.000632

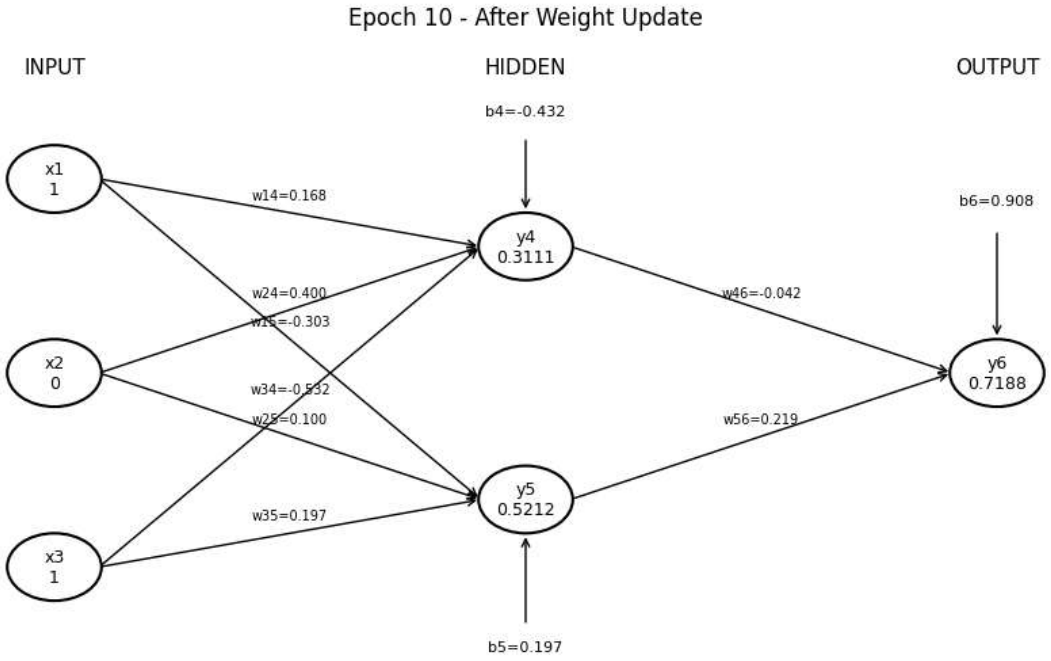
Δw15 = 0.002456
Δw25 = 0.000000
Δw35 = 0.002456
Δb5 = 0.002456

********** UPDATING WEIGHTS **********

Updated weights:
w14 = 0.167669
w24 = 0.400000
w34 = -0.532331
w15 = -0.302533
w25 = 0.100000
w35 = 0.197467
w46 = -0.041742
w56 = 0.219064

Updated biases:
b4 = -0.432331
b5 = 0.197467
b6 = 0.907526

Generating updated graph...



```
================================================
 TRAINING COMPLETED
================================================
Total epochs completed : 10

Target value : 1.000000
Final predicted output : 0.718846
Final error : 0.281154

================================================
 ERROR FOR ALL 10 EPOCHS
================================================
Epoch  1 : Error = 0.526111
Epoch  2 : Error = 0.484752
Epoch  3 : Error = 0.447127
Epoch  4 : Error = 0.413518
Epoch  5 : Error = 0.383823
Epoch  6 : Error = 0.357723
Epoch  7 : Error = 0.334811
Epoch  8 : Error = 0.314665
Epoch  9 : Error = 0.296895
Epoch 10 : Error = 0.281154

================================================
 FINAL COMPARISON
================================================
Expected output : 1.000000
Predicted output: 0.718846
Difference      : 0.281154
```

RESULTS AND DISCUSSION

The unsupervised learning experiment was performed on 500 customer records using four clustering algorithms. K-Means produced five clusters with a Silhouette Score of 0.505, followed by K-Medoids with 0.467 and Hierarchical Clustering with 0.430, The dendrogram provided a visual representation of cluster merging in hierarchical clustering. DBSCAN identified 12 noise points and grouped the remaining 488 records into a single cluster with the selected parameters, so a Silhouette Score was not reported.

In the neural network experiment, the absolute prediction error decreased from 0.526111 in the first epoch to 0.281154 in the tenth epoch. The final predicted output was 0.718846 against a target output of 1.000000. These results demonstrate how repeated weight and bias updates can reduce prediction error during neural network training.

CONCLUSION

This mini project provided practical exposure to unsupervised learning and artificial neural networks using Python. Among the four clustering algorithms, **K-Means performed the best**, achieving the highest Silhouette Score and demonstrating effective customer segmentation based on Age, Annual Income and Spending Score. K-Medoids, Hierarchical Clustering and DBSCAN provided alternative approaches to identifying customer groups and detecting noise. The hierarchical clustering experiment illustrated the cluster-merging process through a dendrogram. The neural network experiment demonstrated forward propagation, backpropagation and iterative weight updates, resulting in a reduction in prediction error over 10 epochs. Overall, the project strengthened the understanding of clustering techniques, data visualization, cluster evaluation and the basic learning mechanism of neural networks.

REFERENCES:

1. MacQueen, J. (1967). “Some Methods for Classification and Analysis of Multivariate Observations.” *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, Vol. 1, pp. 281–297.
2. Kaufman, L., and Rousseeuw, P. J. (1990). *Finding Groups in Data: An Introduction to Cluster Analysis*. John Wiley & Sons.
3. Murtagh, F., and Contreras, P. (2012). “Algorithms for Hierarchical Clustering: An Overview.” *WIREs Data Mining and Knowledge Discovery*, 2(1), 86–97.
4. Ester, M., Kriegel, H.-P., Sander, J., and Xu, X. (1996). “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise.” *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, pp. 226–231.
5. Haykin, S. (2009). *Neural Networks and Learning Machines*. 3rd ed., Pearson.
6. Pedregosa, F., et al. (2011). “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, 12, 2825–2830. Available at: https://jmlr.org/papers/v12/pedregosa11a.html
7. Harris, C. R., et al. (2020). “Array Programming with NumPy.” *Nature*, 585, 357–362. Available at: https://doi.org/10.1038/s41586-020-2649-2
8. The pandas development team. *pandas Documentation*. Available at: https://pandas.pydata.org/docs/
9. Matplotlib Development Team. *Matplotlib Documentation*. Available at: https://matplotlib.org/stable/
10. SciPy Community. *SciPy Hierarchical Clustering and Dendrogram Documentation*. Available at: https://docs.scipy.org/doc/scipy/reference/cluster.hierarchy.html